AI ENGINEERING | ASSIGNMENT 1

Building a Small GPT from Scratch

A 3.2M-parameter Transformer language model trained on Tiny Shakespeare, explained, tested, evaluated, and packaged

Team Members

Hafsa Rashid: 23K-0064
Adina Faraz: 23K-0008
Muneeb ur Rehman: 23K-0038

Instructor: Mr. Farhan Feroze
FAST NUCES, Karachi | Fall 2026

Code: github.com/adeena43/Small-Language-Model
Live Demo: https://adeena43-slm.hf.space
Medium Article: medium.com/@hf.rashid01/building-a-small-gpt-from-scratch-afb5f7720148

1. Problem and Scenario

Consider a small organization with a private collection of text. Although this collection may be insufficient to train a model comparable in scale to ChatGPT, it may contain enough information for a compact language model to learn the vocabulary, writing style, and common patterns of a specific domain.

This leads to the central research question of the project:

Can a small Transformer trained from scratch learn the next-token patterns of a domain-specific corpus well enough to generate coherent text?

Tiny Shakespeare was selected as the domain corpus for this project. The objective was not to build the largest possible model, but to develop a compact GPT-style model whose components could be understood, thoroughly tested, reproduced from a clean setup, and clearly explained.

The complete pipeline is:

Raw text → Tokenizer → Token IDs / Batches → Token Embedding + Positional Encoding → 4 Causal Transformer Blocks → Final LayerNorm → Vocabulary Logits → Next-Token Prediction → Generated Text

This pipeline provides a complete view of the process of building and evaluating a small language model, from raw-text preprocessing to the generation of new sequences.

2. Transformer Foundations

2.1 Why Transformers? Sequential vs. Parallel Processing

A Recurrent Neural Network (RNN) processes a sequence one token at a time. At step t, its hidden state depends on the hidden state from step t-1. Therefore, processing a sequence of T tokens requires T sequential operations.

Self-attention follows a different approach. Rather than processing tokens sequentially, it compares each token with the other tokens through matrix operations. This allows the GPU to perform many operations in parallel, making Transformers highly effective on modern hardware.

However, this advantage comes with a memory cost. Self-attention generates a T × T attention score matrix for each attention head. Consequently, the memory required to store these pairwise scores increases quadratically with the sequence length.

Table 1: Complexity Comparison

Property	RNN	Self-Attention
Sequential operations	O(T)	O(1)
Compute per layer	$O(T \cdot d^2)$	$O(T^2 \cdot d)$
Memory for pairwise scores	None	$O(T^2)$ per head
Longest path between two tokens	O(T)	O(1)

To validate this theoretical comparison experimentally, the forward-pass time of an RNN cell loop and a causal self-attention layer was measured. The experiment used a batch size of 8, d_model = 128, and 4 attention heads on a Tesla T4 GPU.

Table 2: Measured Forward Time on a Tesla T4

Sequence Length T	RNN Loop (ms)	Self-Attention (ms)	Attention Matrix Memory
32	2.29	0.41	0.12 MB
64	4.63	0.74	0.50 MB
128	9.81	0.41	2.00 MB
256	18.67	0.84	8.00 MB
512	37.30	2.43	32.00 MB

The results show that RNN execution time increased almost proportionally with sequence length, rising from 2.29 ms at T = 32 to 37.30 ms at T = 512. In comparison, self-attention remained relatively fast, increasing from 0.41 ms to 2.43 ms. At T = 512, self-attention was approximately 15 times faster than the RNN in this experiment.

The results also demonstrated the expected memory trade-off. Attention matrix memory increased from 0.12 MB to 32 MB, meaning that memory usage became four times larger whenever the sequence length doubled.

This difference arises because an RNN relies on a chain of dependent operations, limiting the GPU's ability to process them in parallel. Self-attention instead converts the sequence into matrix operations that can be executed concurrently. However, this approach introduces an $O(T^2)$ memory requirement, which remains a major challenge when working with longer context lengths.

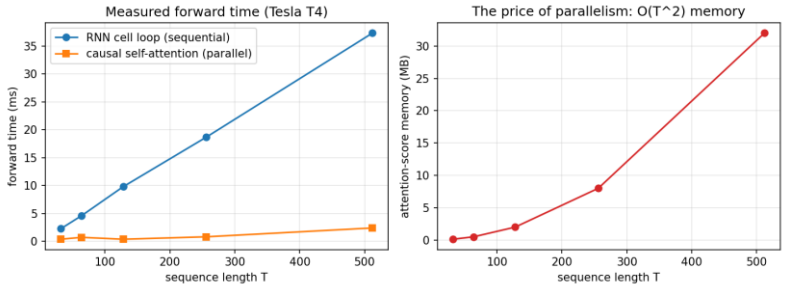


Figure A: Measured forward time (left) and attention-score memory (right) on a Tesla T4.

2.2 Self-Attention: Q, K, V and Scaling

In self-attention, each token representation is projected into three vectors:

- **Query (Q):** Represents what the current token is looking for.
- **Key (K):** Represents the information a token can offer.
- **Value (V):** Contains the information passed to the output.

The attention operation is defined as:

$$Attention(Q, K, V) = softmax(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

The calculation involves several steps. First, $Q \cdot K^T$ produces a score for every query-key pair. This score indicates the relevance of token j to token i.

Next, the scores are divided by $\sqrt{(d_k)}$. This scaling keeps their magnitudes within a suitable range as the dimensionality of the key vectors increases. Without scaling, the dot products may become too large, causing the softmax distribution to become excessively sharp and producing very small gradients.

Softmax is then applied row-wise to convert the scores into attention weights. The weights in each row sum to 1.

Finally, the attention weights are multiplied by V. This produces a weighted combination of the value vectors, allowing each token to incorporate information from the tokens it considers relevant.

2.3 Hand Calculation of Attention

Before relying entirely on PyTorch, a small attention example was worked through manually to verify both the underlying understanding and the implementation. The example contains three tokens.

The complete handwritten working is presented below in Figure 1, across pages 1 to 5.

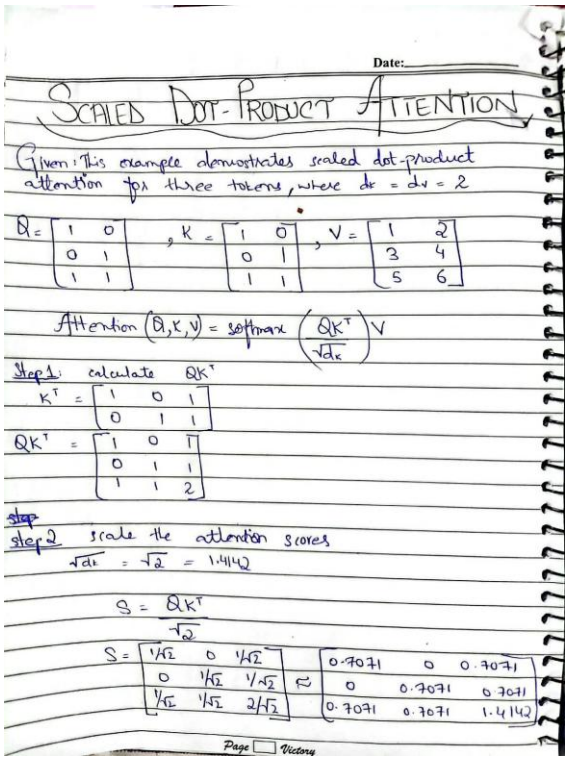


Figure 1 (page 1 of 5): Handwritten calculation showing the given matrices, Q · K^T, and scaling.

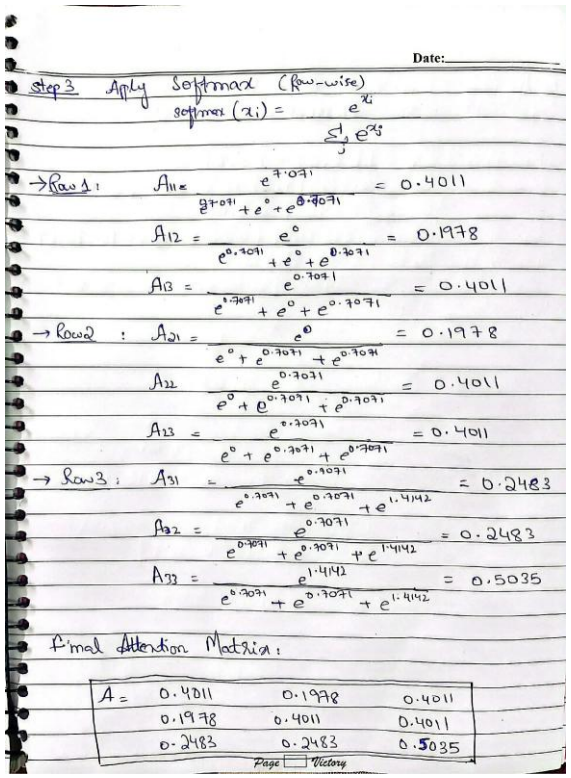


Figure 1 (page 2 of 5): Handwritten calculation showing row-wise softmax.

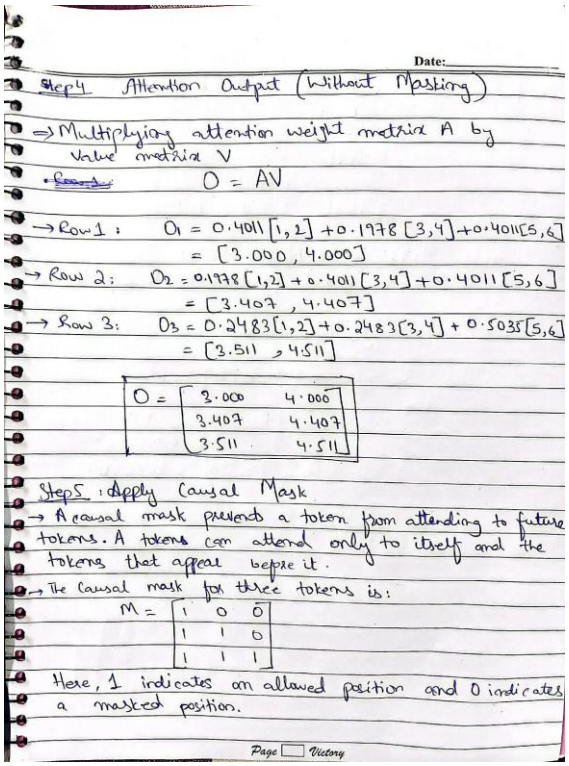


Figure 1 (page 3 of 5): Handwritten calculation showing the output A · V and the causal mask.

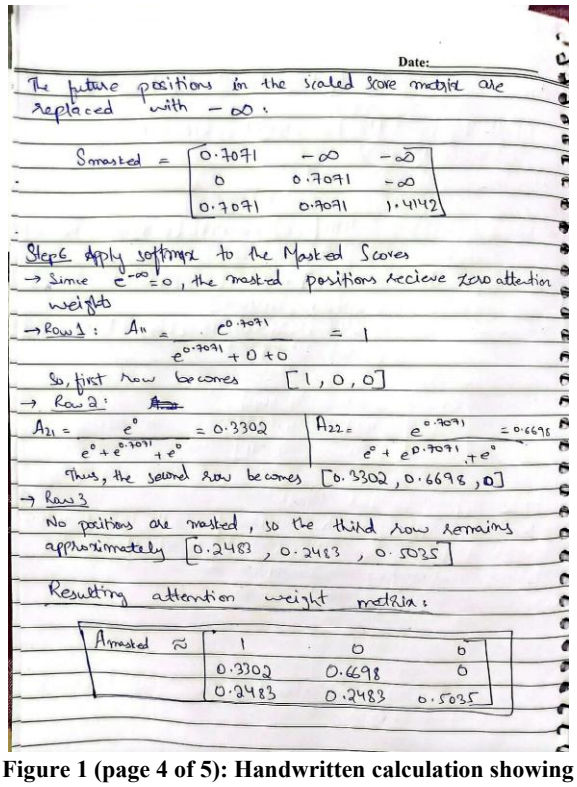


Figure 1 (page 4 of 5): Handwritten calculation showing masked scores and masked softmax.

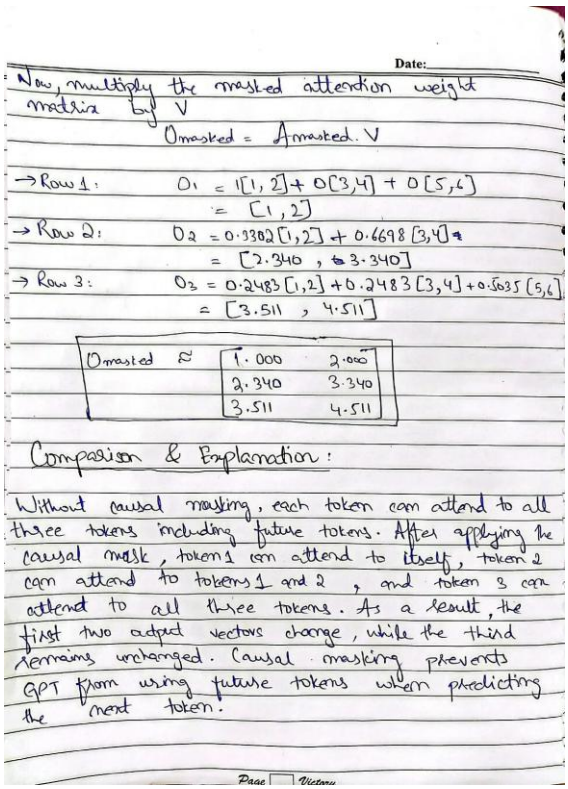


Figure 1 (page 5 of 5): Handwritten calculation showing the masked output and its explanation.

The experiments/hand_calculation.py script reproduces these calculations and confirms that the scaled_dot_product_attention function produces the same results. A unit test also compares the results against an independent NumPy implementation.

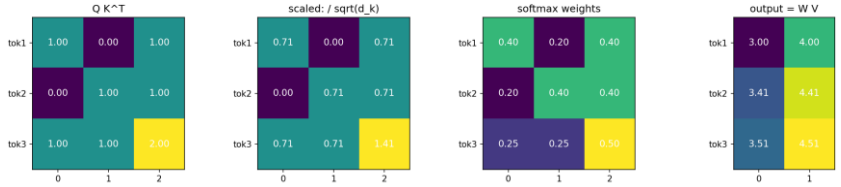


Figure 1b: The same calculation reproduced by the code (experiments/hand_calculation.py).

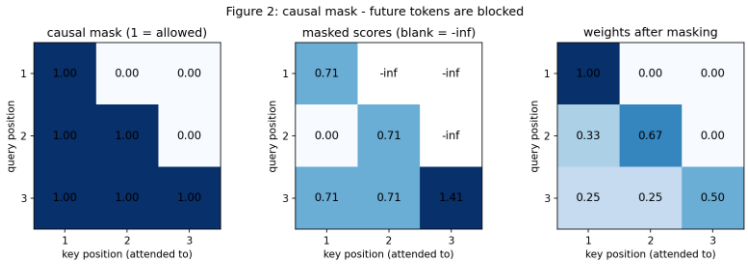


Figure 2: The causal mask blocks future positions, assigning them exactly zero attention weight.

2.4 Multi-Head Attention and Tensor Shapes

A single attention head may focus on one type of relationship between tokens. Multi-head attention allows the model to learn several types of relationships simultaneously.

In the implementation, d_model is divided into H smaller attention heads. Attention is calculated independently for each head. Their outputs are then concatenated and passed through an output projection W_o.

The small example uses a batch size of B = 2, sequence length of T = 6, d_model = 32, H = 4 attention heads, and head dimension $d_h = 8$.

Table 3: Multi-Head Attention Shape Trace

Step	Shape	Operation
Input x	(2, 6, 32)	(B, T, D)
Q, K, V after Linear	(2, 6, 32)	Three D → D projections
Split into heads	(2, 4, 6, 8)	View and transpose
Attention weights	(2, 4, 6, 6)	One T × T matrix per head
Per-head output	(2, 4, 6, 8)	Weights × V
Merge	(2, 6, 32)	Concatenate heads
Output projection W_o	(2, 6, 32)	Linear D → D

The implementation explicitly handles the splitting and merging operations through split_heads and merge_heads in src/attention.py.

```
MULTI-HEAD ATTENTION SHAPE TRACE   (B=2, T=6, d_model=32, n_heads=4, head_dim=8)
  input x (B, T, D)                  (2, 6, 32)
  q/k/v after Linear (B, T, D)       (2, 6, 32)
  split heads q (B, H, T, d_h)       (2, 4, 6, 8)
  attention weights (B, H, T, T)     (2, 4, 6, 6)
  per-head output (B, H, T, d_h)     (2, 4, 6, 8)
  merged / concatenated (B, T, D)    (2, 6, 32)
  after output projection (B, T, D)  (2, 6, 32)
W_q, W_k, W_v, W_o each: Linear(32 -> 32);  concatenating 4 heads of size 8 gives 32 again.
```

Figure 3: Multi-head tensor-shape printout.

A unit test verifies that merge_heads(split_heads(x)) returns the original tensor exactly. This confirms that the reshaping process preserves the underlying information without altering or scrambling it.

2.5 Positional Encoding

Self-attention does not inherently understand token order. If the tokens are shuffled, their corresponding outputs are shuffled as well. Without positional information, the model would therefore struggle to distinguish between sentences such as "dog bites man" and "man bites dog". To represent token order, a positional signal is added to each token embedding.

The model uses the sinusoidal positional encoding introduced in the original Transformer paper:

$$PE(pos, 2i) = sin(pos / 10000^{(2i / d_model)})$$

$$PE(pos, 2i+1) = cos(pos / 10000^{(2i / d_model)})$$

Each dimension uses a different frequency. Some dimensions change rapidly with position, while others change more gradually. Together, these values form a distinct pattern for each position.

The values remain between −1 and 1, and the encoding does not require trainable parameters. It is stored as a fixed buffer.

Learned positional embeddings are also implemented as an optional alternative.

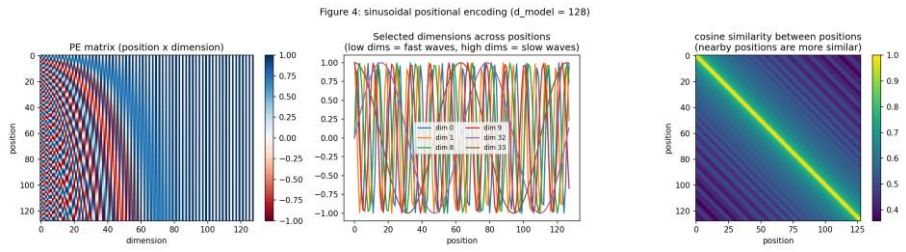


Figure 4: Sinusoidal positional encoding.

2.6 The Transformer Block

The decoder block follows the pre-LayerNorm design used by GPT-2:

$$x = x + MultiHeadAttention(LayerNorm(x), causal_mask)$$

$$x = x + FeedForward(LayerNorm(x))$$

$$FeedForward(x) = Linear(4d \rightarrow d)(\ GELU(\ Linear(d \rightarrow 4d)(x)\ )\ )$$

Several components are important to this design. Residual connections provide a direct path for gradients through the network. They allow each layer to learn a correction to the existing representation instead of having to learn an entirely new representation.

LayerNorm maintains a stable scale for token representations. LayerNorm is applied before each sub-layer, following the pre-LN design, which generally supports more stable training.

The attention layer enables information exchange between different positions in the sequence, while the feed-forward network processes each position independently.

The feed-forward network accounts for a substantial portion of the parameters in each block: 525,568 out of 789,760 parameters.

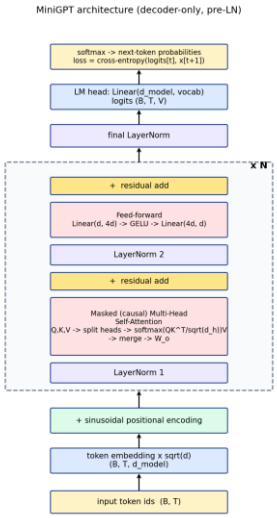


Figure B: MiniGPT architecture, from token IDs to logits.

2.7 BERT (Masked LM) vs. GPT (Causal LM)

BERT and GPT both use Transformer blocks, but they are trained with different objectives. Their main difference lies in how attention is applied.

Property	BERT (Masked LM)	GPT (Causal LM)
Example input	to [MASK] or not [MASK] be	to be or not to
Target	Hidden words: "be", "to"	Next token at every position
Attention	Bidirectional	Causal
Visibility	Left and right	Only previous tokens
Main purpose	Language understanding	Left-to-right generation

BERT can attend to tokens on both sides because it is trained to predict masked words. GPT, however, predicts the next token and can therefore use only information from the left.

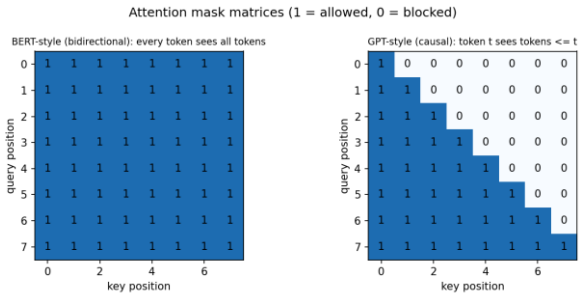


Figure C: Attention masks. BERT allows every token to attend to every token, whereas GPT blocks future positions.

To demonstrate the importance of causal masking, the same model was trained for 800 steps with and without the mask.

With the causal mask, the validation loss reached 1.98, following a normal learning curve. Without the mask, the loss dropped to 0.04 after approximately 220 steps. Although this appears to indicate excellent performance, it actually shows that the model is accessing information it should not have.

Without the mask, the model can see future tokens and effectively obtain the answer from its input. As a result, it appears to perform extremely well during training but cannot be relied upon for valid next-token generation.

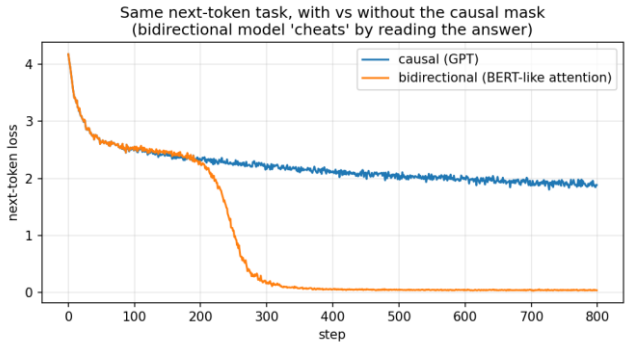


Figure D: Next-token training with and without the causal mask.

In addition, tokens 12 to 15 were modified and the resulting changes in outputs at positions 0 to 7 were measured. With the causal mask, the change was exactly 0. Without the mask, the outputs changed by 4.2×10^{-4}.

This experiment demonstrates the necessity of causal masking in GPT and explains why BERT uses a different training objective.

3. Implementation

The project was implemented using fundamental PyTorch components, including tensors, nn.Linear, nn.LayerNorm, softmax, autograd, and AdamW. The Transformer and GPT components were implemented as part of this project.

The main source files are listed below.

File	Purpose
src/tokenizer.py	Character-level tokenizer with encode, decode, save, and load functionality
src/dataset.py	Downloads the dataset, creates the training/validation split, and generates shifted batches
src/attention.py	Implements the causal mask, scaled dot-product attention, single-head attention, and multi-head attention
src/positional_encoding.py	Implements sinusoidal encoding and optional learned positional encoding
src/transformer_block.py	Contains the feed-forward network and pre-LN Transformer block
src/gpt.py	Implements the complete model, loss calculation, generation, and model summary
src/train.py	Handles the training loop, learning-rate schedule, logging, checkpoints, resuming, and overfitting test
src/generate.py, src/api.py	Provide command-line generation and the FastAPI web service

3.1 Tokenizer

A character-level tokenizer was implemented using the sorted set of unique characters in the corpus, resulting in a vocabulary of 65 tokens.

The vocabulary contains 52 uppercase and lowercase letters, the space character, the newline character, and 11 punctuation symbols. The only digit present in the dataset is "3".

Sorting the characters makes the tokenizer deterministic, ensuring that the same input text always produces the same token IDs.

No special tokens are used, such as padding, start-of-sequence, or end-of-sequence tokens. Since training uses fixed-length windows from a continuous stream of text, padding is unnecessary.

If a user enters a character outside the vocabulary, such as most digits or emojis, the API replaces it with a space and reports which characters were replaced.

Character-level tokenization was selected because its vocabulary is small, containing 65 tokens compared with the tens of thousands commonly used by subword tokenizers. This keeps the embedding and output layers compact and allows a larger proportion of the parameter budget to be allocated to the Transformer blocks.

Its main limitation is that the model must learn spelling patterns at the character level. This contributes to the invented words observed in the generated samples.

3.2 Model Configuration

Table 4: Final Model Configuration

Parameter	Value	Reason
vocab_size	65	Determined by the tokenizer
context_length	128	Assignment default; sufficient for approximately 2 to 3 lines of verse
d_model	256	Upper option from the handout and practical to train on a T4
n_heads	4	Head dimension is 64
n_layers	4	Assignment default
ffn_dim	1024	4 × d_model
dropout	0.1	Provides regularization
Positional encoding	Sinusoidal	Required by the assignment
Weight tying	Off	Retained as an optional experiment

Two design choices from GPT-2 were also incorporated. First, the token embeddings are multiplied by √(d_model) before the positional encoding is added.

Second, the output projections of the residual branches are initialized using a smaller standard deviation of $0.02 / \sqrt{(2 \cdot n_layers)}$. This helps prevent the residual stream from growing excessively as the number of layers increases.

3.3 Attention and Masking in Code

The main attention implementation is shown below:

```
def causal_mask(T):
    return torch.tril(torch.ones(T, T, dtype=torch.bool))  # True = may attend


def scaled_dot_product_attention(q, k, v, mask=None):
    scores = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
    if mask is not None:
        scores = scores.masked_fill(~mask, float("-inf"))
    weights = torch.softmax(scores, dim=-1)
    return weights @ v, weights
```

The causal mask is applied before softmax by setting blocked positions to $-\infty$. If attention weights were set to zero after softmax, the values in each row would no longer sum to 1. Similarly, replacing blocked scores with 0 instead of $-\infty$ would not fully prevent future positions from receiving attention, because $e^0 = 1$.

3.4 Training Loop

The training process follows these steps:

1. Sample 64 random windows of 128 characters from the training split.
2. Shift each input sequence by one position to create the target, using target[t] = input[t + 1].
3. Perform a forward pass to produce logits of shape (64, 128, 65). Cross-entropy loss is calculated across all 64 × 128 positions, providing 128 next-token prediction signals per sequence.
4. Perform backpropagation, clip gradients at 1.0, and update the parameters using AdamW.
5. Every 250 steps, calculate training and validation loss and save the best checkpoint.
6. Every 1,000 steps, generate a sample using "ROMEO:" as the prompt.

The learning rate uses linear warm-up during the first 100 steps, reaching 3×10^{-4}, followed by cosine decay to 3×10^{-5}.

Weight decay is set to 0.01 for weight matrices, while biases and LayerNorm parameters are excluded. Because Google Colab sessions may disconnect, the trainer saves a resume file every 250 steps. This file contains the model, optimizer, random-number state, and logs, allowing training to resume from the most recently saved point.

4. Dataset and Preprocessing

The Tiny Shakespeare dataset was used.

Item	Value
Source	Tiny Shakespeare (Hugging Face card karpathy/tiny_shakespeare; raw copy from Karpathy's char-rnn repository)
Size	1,115,394 characters, 40,001 lines, approximately 1.1 MB
Tokenizer	Character-level, 65-token vocabulary, no special tokens
Split	First 90% for training, last 10% for validation
Context length	128 characters
Filtering	None; the text is used exactly as downloaded

The first 90% of the text contains 1,003,854 tokens, while the remaining 10% contains 111,540 tokens. A contiguous split was used instead of randomly selecting windows for the two sets. Random windows could cause overlapping portions of the same lines to appear in both the training and validation data, making the validation loss appear better than the model's actual performance.

By maintaining a contiguous split, the validation data comes from scenes that were not encountered during training.

The dataset is not committed to Git. Instead, src/dataset.py downloads it automatically when required for the first time.

5. Training Setup

Setting	Value
Hardware	Google Colab, NVIDIA Tesla T4 (16 GB)
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay = 0.01
Learning rate	3×10^{-4} peak, 100 warm-up steps, cosine decay to 3×10^{-5}
Batch size	64 sequences × 128 characters = 8,192 tokens per step
Training steps	5,000
Gradient clipping	1.0
Evaluation	Every 250 steps, using 20 random batches for training and validation
Checkpoints	best.pt, last.pt, and resume file
Seed	1337 for Python, NumPy, and PyTorch
Training time	380.6 s (6.3 minutes)
Throughput	Approximately 110,800 tokens/second
Peak GPU memory	794 MB

The final model contains 3,192,832 parameters (3.19M). The parameter count is distributed as follows: token embedding, 16,640; positional encoding, 0 (fixed buffer); each of the four Transformer blocks, 789,760 (attention, 263,168; feed-forward network, 525,568; LayerNorms, 1,024); final LayerNorm, 512; and language-model head, 16,640.

As a sanity check, an untrained model with 65 possible characters should have a loss close to ln(65) = 4.17. The untrained model produced a loss of 4.21, which is close to the expected value.

```
MiniGPT model summary
============================================================
vocab_size        : 65
context_length    : 128
d_model           : 256
n_heads           : 4  (head_dim = 64)
n_layers          : 4
ffn_dim           : 1024
dropout           : 0.1
position encoding : sinusoidal
weight tying      : False
------------------------------------------------------------
token embedding   :     16,640
position encoding :          0  (fixed buffer, no parameters)
each block (x4)   :    789,760
  - attention     :    263,168
  - feed-forward  :    525,568
  - 2 LayerNorms  :      1,024
final LayerNorm   :        512
LM head           :     16,640
------------------------------------------------------------
TOTAL parameters  :  3,192,832  (3.19M)

forward check: idx (2, 128) -> logits (2, 128, 65), initial loss 4.212 (uniform guess = ln 65 = 4.174)
```

Figure 5: Model summary and parameter count.

5.1 Debugging Gate: Overfitting a Tiny Batch

Before the full training run, the model was tested on a small, fixed batch. The model was trained on 4 sequences for 300 steps with dropout disabled. If gradient computation, target shifting, and causal masking were implemented correctly, the model should be able to memorize this batch.

The loss decreased from 4.25 to 0.0002, with most of the improvement occurring within the first 50 steps. This experiment is also part of the automated test suite. It provided a useful debugging check before the longer training run.

6. Results

6.1 Final Model: Size and Training Time

Item	Result
Parameters	3,192,832 (3.19M)
Training time	380.6 s (6.3 minutes)
Final train loss	1.367
Final validation loss	1.586
Best validation loss	1.585
Validation perplexity	4.9
Full validation loss	1.591
Full validation perplexity	4.91
Bits per character	2.30

6.2 Training and Validation Loss

Both training and validation loss decreased rapidly during the first 1,000 steps, followed by more gradual improvements.

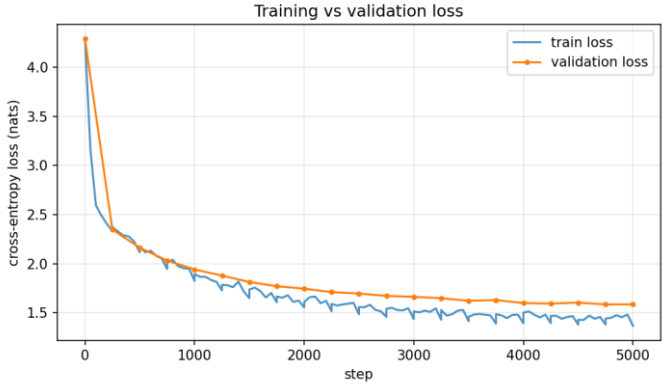


Figure 6: Training vs. validation loss over 5,000 steps.

Step	Train Loss	Validation Loss	Validation Perplexity
0	4.290	4.289	72.9
1,000	1.823	1.941	7.0
2,000	1.555	1.746	5.7
3,000	1.437	1.663	5.3
4,000	1.394	1.599	4.9
5,000	1.367	1.586	4.9

The validation loss continued to decrease until the end of training, reaching its best value of 1.585 at step 5,000.

The difference between the final training and validation losses was approximately 0.22. This suggests that the model was continuing to learn rather than merely memorizing the training set. The gradually increasing gap indicates mild overfitting, which is expected with a small dataset. Dropout and weight decay help control this effect.

When evaluated on the complete validation split of 111,488 characters, the model achieved a loss of 1.591, a perplexity of 4.91, and 2.30 bits per character. A perplexity of 4.9 means that the model is, on average, approximately as uncertain as if it were choosing between five equally likely characters. At the beginning of training, it had roughly 65 possible characters to choose from.

6.3 Generated Samples

The following samples were generated using the final checkpoint, with a temperature of 0.8 and 200 new characters for each of five fixed prompts.

Sample 1: Prompt "ROMEO:"

```
ROMEO:
You have now off, which you have senatory
Of that Oxford called him and him us her to be
Methink, endream, you fall to my more own procful
I can this now in a griefe-sent in the sea,
Who thou dest no
```

Sample 2: Prompt "To be, or not to be"

```
To be, or not to be man!


Provost:
I am am a presued on him.


SICINIUS:
Why you love's not
To this deserved sword to again Warwick,
I prince to thee, and high him he poor of his but
To blood the fire; sir. Who fit thy f
```

Sample 3: Prompt "KING HENRY: / What say you"

```
KING HENRY:
What say you are the paint of York and friend!


QUEEN ELIZABETH:
That he may in the cause no much soldier for his need.


QUEEN ELIZABETH:
Madam, so a changed-speak between throaty who
must he wilt welcome, with c
```

Sample 4: Prompt "The future of artificial intelligence"

```
The future of artificial intelligence; for yours
Do might. And as you all the sorrow and heavy.


LEONTES:
You would knock my soul in the world Richard:


ROMEO:
The firest thou wife that I
```

Sample 5: Prompt "First Citizen: / Before we proceed"

```
First Citizen:
Before we proceed me to commons, this here's death,
The milk but brother'd with the ever doom,
And the rest the subjection that wilt of griet be
All thee slake of the hands part.


BENVOLIO:
You the glory fly thee, or
```

Figure 7: Generation samples from five fixed prompts.

What the Model Gets Right

The generated text demonstrates that the model learned several structural patterns from Shakespeare. It places speaker names on separate lines in capital letters followed by a colon, uses real character names such as QUEEN ELIZABETH, LEONTES, and BENVOLIO, follows Shakespearean line formatting, uses period-specific words such as "thou," "thee," and "wilt," and generates a substantial proportion of real English words.

What the Model Gets Wrong

The main limitation is that the generated text often lacks clear meaning and consistent context. The model sometimes produces invented words, including "senatory," "procful," and "griefe-sent." It also combines characters and settings from different plays. For example, characters associated with the King Henry plays may appear alongside Romeo and Benvolio.

The out-of-domain prompt "The future of artificial intelligence" also quickly shifts into Shakespeare-style text. This occurs because the model was trained exclusively on Shakespeare's works.

The changes in generation as training progresses can also be observed:

```
step 0:    ROMEO:?qC;xbDkRZkNdc.wj,ZTxOL1T-eHtK
step 1000: ROMEO: Do hengesives lofe cannter to beast: what good be the not
step 5000: ROMEO: Who let you should shall be of the Montague:
```

At step 0, the output is essentially random. By step 1,000, the model has started learning word shapes and speaker names, although many words remain incorrect. By step 5,000, most generated words are real, and the overall Shakespearean structure is considerably clearer.

6.4 Attention Inspection

To examine how the trained model uses attention, the model was given the prompt "ROMEO:" followed by a new line and "O, that I were a glove upon" (34 characters), and the attention weights of all 16 heads were plotted, representing 4 layers with 4 heads each.

Each heatmap shows how much attention each character assigns to every earlier character.

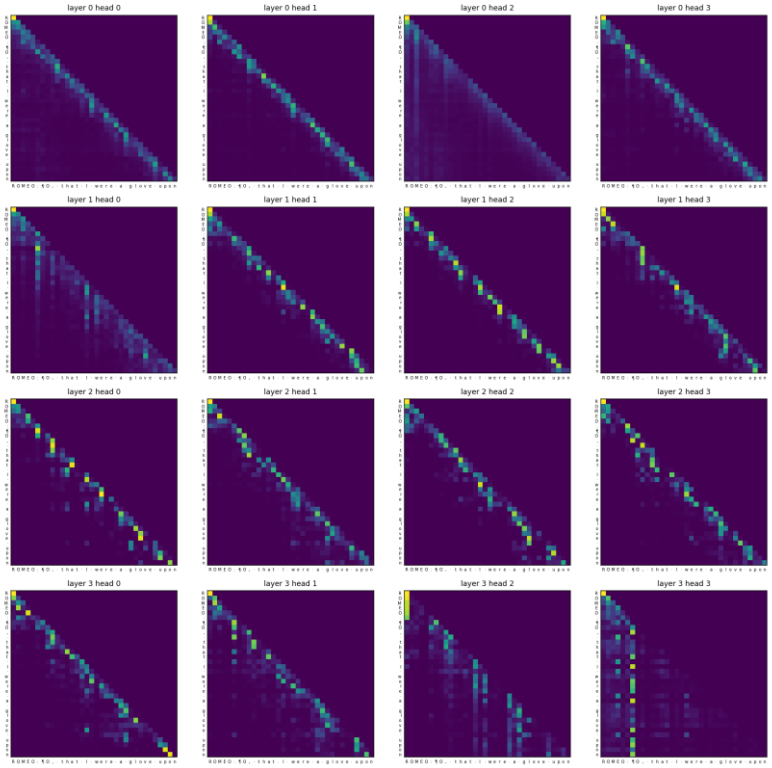


Figure 8: Attention weights of the trained model (rows represent queries and columns represent keys).

Observation

All attention maps remain lower-triangular, confirming that the causal mask continues to function correctly after training.

Most heads, particularly in the first layer, assign significant attention to the current character and the previous one or two characters. This is reasonable for a character-level model because nearby characters provide useful information for learning spelling patterns.

Some heads display broader attention patterns. For example, Layer 0 Head 2 distributes its attention across a larger portion of the preceding context.

In deeper layers, some heads form vertical stripes, indicating that multiple characters attend to the same earlier positions. Layer 3 Head 3 assigns considerable attention to a single earlier position: the line break right after the speaker name "ROMEO:".

Interpretation

The attention patterns suggest that different heads have learned different roles. Some focus primarily on local character patterns, while others appear to use a broader context.

The strong attention assigned to the line break after the speaker name suggests that this head keeps track of where the current speech begins, which is useful for following the speaker-and-line layout of the plays.

However, these specific interpretations cannot be confirmed from the heatmaps alone. The visualization clearly demonstrates that the attention heads have developed different patterns of behaviour.

7. Ablation Study

For the ablation study, one factor was varied at a time while keeping the remaining settings constant. Each run used the same random seed, dataset, optimizer, learning-rate schedule, 2,000 training steps, and batch size of 64. To fit all nine experiments on a free GPU, a smaller base model was used with d_model = 128, ffn_dim = 512, 4 layers, 4 heads, and a context length of 128, containing approximately 0.81M parameters.

Because this configuration differs from the main 3.2M-parameter model, the ablation results should not be directly compared with the results of the main model.

In addition to validation loss, the valid word rate was measured, which represents the percentage of generated words that appear somewhere in the training text.

Table 5: Ablation Results

Run	Parameters	Val Loss	Perplexity	Train Time (s)	Peak GPU (MB)	Valid Words
2 layers	413,440	2.185	8.89	43.4	261	35%
4 layers	809,984	2.124	8.37	81.2	470	44%
6 layers	1,206,528	2.053	7.79	119.2	678	52%
2 heads	809,984	2.129	8.41	71.0	398	37%
4 heads	809,984	2.124	8.37	81.5	470	44%
8 heads	809,984	2.108	8.23	102.5	614	48%
Context 64	809,984	2.100	8.16	42.6	213	51%
Context 128	809,984	2.124	8.37	81.7	470	44%
Context 256	809,984	2.163	8.70	194.7	1,201	45%

7.1 Depth: 2 vs. 4 vs. 6 Layers

Increasing the number of Transformer layers produced a clear improvement in these experiments.

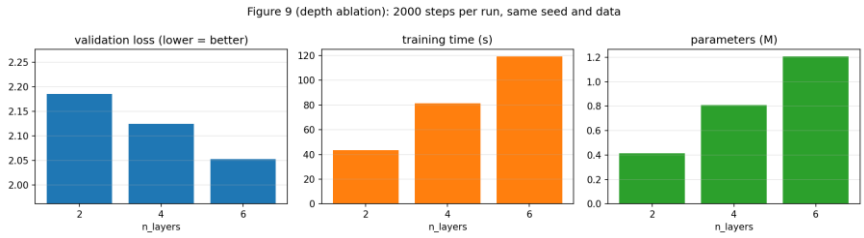


Figure 9a: Depth ablation showing validation loss, training time, and parameter count.

Validation loss decreased from 2.185 with 2 layers to 2.124 with 4 layers and 2.053 with 6 layers. The valid word rate also increased from 35% to 52%.

However, this improvement came with higher computational costs. The 6-layer model required approximately 2.7 times more training time than the 2-layer model.

Among the three factors tested, depth had the strongest effect. Each additional Transformer block provides another stage for mixing information across positions and processing the resulting representations.

For a character-level model, deeper layers may help develop increasingly complex patterns, progressing from individual characters to character sequences and word-level structures.

Since performance continued to improve with six layers, increasing the depth further may provide additional benefits, although it would also increase training time and memory usage.

7.2 Heads: 2 vs. 4 vs. 8

Changing the number of attention heads does not change the total parameter count because the same d_model is divided among a different number of heads.



Figure 9b: Heads ablation.

Validation loss changed only slightly, from 2.129 with 2 heads to 2.124 with 4 heads and 2.108 with 8 heads. The valid word rate increased from 37% to 48%.

However, computational costs also increased. Training time rose from 71 seconds to 103 seconds, while memory usage increased from 398 MB to 614 MB.

A larger number of attention heads allows the model to focus on different patterns simultaneously, which may explain the small improvement. However, the difference in validation loss was only 0.02, and each configuration was evaluated once using a single random seed. Therefore, this difference may partly reflect random variation.

The more consistent observation is that increasing the number of heads increases both memory usage and training time.

7.3 Context Length: 64 vs. 128 vs. 256

With the fixed training budget of 2,000 steps, the shortest context length achieved the lowest validation loss: 2.100 for context 64, 2.124 for context 128, and 2.163 for context 256.

Longer contexts were also considerably more expensive. Training time increased from 43 seconds to 195 seconds, while peak GPU memory rose from 213 MB to 1,201 MB.



Figure 9c: Context length ablation.

These results require careful interpretation because they do not establish that shorter context lengths are generally better.

First, the batch size was kept fixed at 64 sequences. Therefore, longer contexts process more tokens per step, even though all configurations receive the same number of training steps.

Second, each model is evaluated using windows corresponding to its own context length. One possible explanation is that, within only 2,000 training steps, the small model is still primarily learning short-range patterns, such as spelling and common character sequences. These patterns may be learned effectively using shorter contexts.

Therefore, this experiment does not establish that context 64 is inherently better than context 128 or 256. A fairer comparison would give each configuration the same number of processed tokens and train the models until their validation losses converge.

However, the experiment clearly demonstrates the $O(T^2)$ computational cost of self-attention. When context length increased fourfold, from 64 to 256, memory usage increased by approximately 5.6 times.

7.4 Greedy Decoding vs. Sampling

Different decoding strategies were also compared using the final trained model. The three-word repeat rate measures the percentage of repeated three-word sequences in the generated output.

Table 6: Greedy vs. Sampling

Strategy	Valid Word Rate	3-Word Repeat Rate
Greedy	97.4%	77.1%
Temperature 0.5	96.0%	0%
Temperature 0.8	90.9%	0%
Temperature 1.2	74.1%	0%
Top-k 10, temperature 0.8	93.8%	0%
Top-p 0.9, temperature 0.8	95.1%	0%

Greedy decoding achieved a high valid-word rate of 97.4%, but it also produced a three-word repetition rate of 77.1%.

In contrast, none of the sampling methods produced three-word repetitions in this experiment. Increasing the temperature introduces more randomness, making the output more diverse but also increasing the number of invented words. Top-k and top-p sampling provide a balance by restricting the selection to likely candidates while retaining some randomness.

The final generation setup uses sampling instead of greedy decoding because it produces more varied text and avoids the strong repetition observed with greedy decoding.

8. Failure Analysis

Failure 1: Lack of Meaning, Invented Words, and Mixed-Up Plays

The generated text often resembles Shakespearean writing but does not maintain a clear meaning. It also contains invented words and combines characters from different plays.

Several likely causes were identified. First, the context length is limited to 128 characters, corresponding to approximately two lines of verse. This restricts the model's ability to retain information about the speaker, scene, or earlier events in longer passages.

Second, character-level tokenization requires the model to learn spelling patterns before developing higher-level language structures.

Third, the model is relatively small, with 3.2 million parameters, and the training corpus is only around 1 MB. This is very limited compared with the resources used to train modern large language models.

Potential improvements include increasing the context length, adopting a subword tokenizer, increasing the model size, and training on a larger dataset.

Failure 2: Greedy Decoding Gets Stuck in Loops

Greedy decoding produced a three-word repetition rate of 77.1%. This occurs because greedy decoding always selects the single most probable next character. Once the model enters a common phrase, repeatedly selecting the most probable continuation can cause it to return to the same phrase.

This issue is addressed using temperature, top-k, and top-p sampling. These strategies introduce controlled randomness and reduce the likelihood of the model becoming trapped in repetitive patterns.

Failure 3: Out-of-Domain Prompts and Unknown Characters

When given the prompt "The future of artificial intelligence", the model quickly shifts back to Shakespeare-style text.

This behaviour is expected because the model was trained exclusively on Shakespeare's works and has no training data covering artificial intelligence or other domains.

The fixed character vocabulary introduces another limitation. Characters outside the 65-token vocabulary, including most digits and emojis, are replaced with spaces.

Therefore, this model should not be treated as a general-purpose text generator or a source of factual information.

Failure 4: Limitations of Single-Run Experiments

Each ablation configuration was evaluated once using a single random seed. In addition, the ablation experiments used a smaller model and only 2,000 training steps.

Consequently, small differences between configurations may fall within normal random variation. For example, the validation-loss difference between 2 and 8 attention heads was only 0.02.

The context-length experiment was also affected by the fixed training-step budget. For more reliable conclusions, each configuration should ideally be evaluated using at least three different random seeds, with every model trained until convergence.

9. Engineering and Packaging

9.1 Tests

The repository contains 31 unit tests, all of which pass. The main tests cover the following areas:

1. **Attention calculation:** The attention implementation is verified to match both the hand-calculated example and an independent NumPy implementation.
2. **Causal correctness:** Tokens after position t are modified to check whether the outputs at positions ≤ t remain unchanged. With the causal mask, the outputs remain exactly the same. A control test without the mask confirms that the test can detect future-information leakage.
3. **Tensor shapes:** The tests verify the expected shapes for attention, multi-head splitting and merging, the Transformer block, and the complete model.
4. **Training and checkpoint functionality:** The tests verify that the model can memorize a small batch, reload a checkpoint in a new process, resume an interrupted training run, and respond correctly through the API.

9.2 Command-Line Inference

Text can be generated from the command line using the following commands:

```
python -m src.generate --prompt "ROMEO:" --max-new-tokens 200 --temperature 0.8
python -m src.generate --prompt "ROMEO:" --top-k 40
python -m src.generate --prompt "ROMEO:" --greedy
python -m src.generate --fixed-prompts
```

Each run displays the prompt, generated text, parameter count, context length, and temperature. This makes the generated output easier to reproduce and interpret.

```
Prompt: 'ROMEO:'
Generated: ROMEO:
You have now off, which you have senatory
Of that Oxford called him and him us her to be
Methink, endream, you fall to my more own procful
I can this now in a griefe-sent in the sea,
Who thou dest no
model parameters: 3.19M
context length: 128
temperature: 0.8
------------------------------------------------------------
```

Figure 10a: CLI inference.

The checkpoint is self-contained and stores the model weights, configuration, and vocabulary. Therefore, no additional file is required to load the trained model.

9.3 REST API

The trained model is packaged as a FastAPI service with four endpoints: GET / for the web page, GET /info, GET /health, and POST /generate.

The API validates incoming requests. It supports prompts of up to 1,000 characters, a maximum of 500 new tokens, temperature values between 0 and 2, and an optional seed for reproducible generation.

Example request:

```
curl -X POST https://adeena43-slm.hf.space/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt": "ROMEO:", "max_new_tokens": 120, "temperature": 0.8}'
```

The model is deployed on Hugging Face Spaces using the Gradio SDK and free ZeroGPU hardware. A small app.py entry point serves the web page at /, along with the same /generate, /health, and /info endpoints. The model itself runs on the CPU.

Live URL: https://adeena43-slm.hf.space

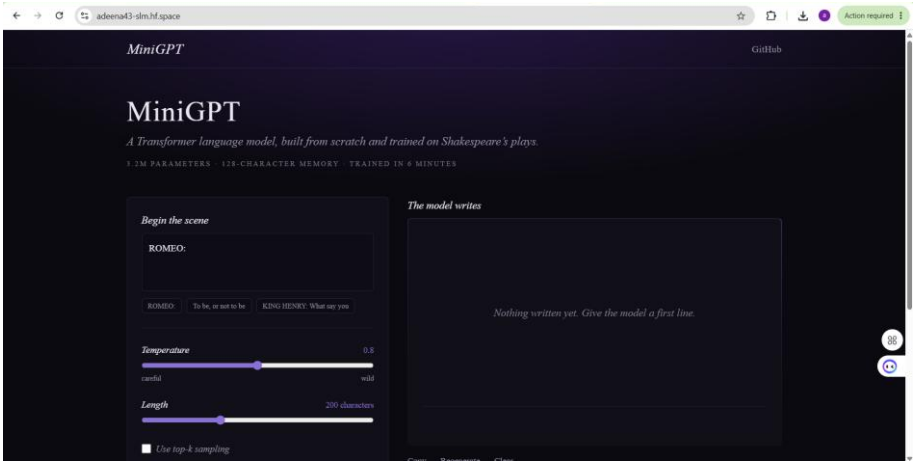


Figure 10b: The deployed web page running on Hugging Face Spaces (adeena43-slm.hf.space).

Free Hugging Face Spaces become inactive after a period without activity. Consequently, the first request after a pause may take longer while the Space restarts.

9.4 Reproducing the Project

The complete project can be reproduced using the following commands:

```
git clone https://github.com/adeena43/Small-Language-Model.git
cd Small-Language-Model
pip install -r requirements-train.txt
python -m pytest -q
python -m src.train --overfit_tiny
python -m src.train --config configs/tiny_gpt.yaml
python experiments/ablation.py --group depth heads context --steps 2000 \
     --set model.d_model=128 model.ffn_dim=512
python -m src.generate --prompt "ROMEO:"
```

A Google Colab notebook is also included, notebooks/colab_demo.ipynb, which runs the complete pipeline on a free GPU. The notebook saves outputs to Google Drive so that progress is retained if the Colab session disconnects.

All experiments use a fixed random seed, and the model configuration is stored in a single YAML file. No secrets or large files were committed to the repository, apart from the 12.8 MB final checkpoint.

10. Conclusion

This project focused on one main question:

Can a small Transformer trained from scratch learn the next-token structure of a narrow domain well enough to generate coherent text?

Based on the results, the answer is yes for structure, but not yet for meaning. The 3.2M-parameter model, trained for approximately six minutes, successfully learned several important characteristics of Shakespearean text. These included vocabulary, spelling patterns, speaker names, line formatting, and the general writing style of the training corpus.

The model reduced its uncertainty from 65 possible characters to approximately five on average. However, it could not consistently maintain a meaningful story, topic, or idea across longer sequences.

The project also provided several important insights into Transformer language models.

- **Attention trades memory for parallelism.** In this experiment, self-attention was approximately 15 times faster than an RNN at a sequence length of 512, but its memory requirement increased quadratically with sequence length.
- **Causal masking is essential for GPT.** Without the causal mask, the model could access future tokens during training, allowing the loss to approach zero without learning useful generative behaviour.
- **Depth had the strongest effect in the ablation study.** Increasing the number of Transformer layers produced the clearest improvement, whereas increasing the number of attention heads provided only a small improvement. Longer context lengths did not provide an advantage within the limited training budget.
- **Testing individual components is important.** The hand-calculated attention example, future-information leakage test, and tiny-batch overfitting test helped verify individual components. These tests could identify implementation problems that might not be apparent from the overall training loss alone.

Overall, building this model from scratch provided practical experience with the complete Transformer language-model pipeline, including tokenization, positional encoding, self-attention, causal masking, Transformer blocks, training, evaluation, generation, testing, and deployment.

Future improvements could include adopting a subword tokenizer, increasing the context length, implementing a KV cache for faster generation, training on a larger corpus, and repeating the ablation experiments using multiple random seeds.

References

1. Vaswani, A. et al. (2017). *Attention Is All You Need.* NeurIPS. Used for attention, multi-head attention, and sinusoidal positional encoding.
2. Radford, A. et al. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI. Used for GPT-2 design choices, including pre-LN blocks and scaled residual initialization.
3. Devlin, J. et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL. Used for masked language modelling concepts.
4. Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). *Layer Normalization.* arXiv:1607.06450.
5. Hendrycks, D. and Gimpel, K. (2016). *Gaussian Error Linear Units (GELUs).* arXiv:1606.08415.
6. Loshchilov, I. and Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR.

7. Karpathy, A. *nanoGPT* and *Let's build GPT: from scratch, in code, spelled out.* Used as conceptual references for the training setup, AdamW configuration, warm-up, cosine learning-rate schedule, and character-level Tiny Shakespeare training.
8. Karpathy, A. *Tiny Shakespeare Dataset.* Available through the Hugging Face dataset repository.
9. Course material: rohitg00/ai-engineering-from-scratch, Phase 07: Transformers Deep Dive, Lessons 01 to 07. Used for conceptual understanding.